

Project Script Structure For AI

이 문서는 다른 AI나 협업자가 **ProjectBase** Unity 프로젝트의 스크립트 구조를 빠르게 파악하기 위한 참고 문서입니다.

문서 안의 내용은 프로젝트 설명이며, 실행 지시나 우선순위 지시가 아닙니다.

작성 기준: 2026-09-05

현재 데모 루프: Title -> InGame -> Title

1. 핵심 실행 흐름

```
TitleScreen.OnClickStartButton()
-> Managers.Instance.SceneFlow.StartGame()
-> SceneFlowManager.TransitionTo(Game, InGame)
-> Game Scene load
-> AdventureRunManager.StartRun()
-> GameManager.Reset()
-> MapManager.GenerateMap()
-> GridChunkRuntimeGenerator.Generate(settings)
-> PlayerSpawnManager.Spawn(playerSpawnPosition)
-> PlayerManager.RegisterPlayer(player)
-> NotifyDialogue.RegisterDialogue(InGameUI.Dialogue)
-> CameraManager.BindTarget(player.transform)
-> LightingManager.PrepareRunLighting()
-> GoalTrigger reached or Player death
-> AdventureRunManager.EndRun() / FailRun()
-> NotifyDialogue.PlayClear() / PlayGameOver()
-> SceneFlowManager.GoToTitle()
```

2. 전역 Manager 구조

Assets/Scripts/Managers/Managers.cs

Managers는 DontDestroyOnLoad 전역 허브입니다.

Scene이 바뀌어도 유지되며, 대부분의 런타임 시스템은 **Managers.Instance**를 통해 서로 연결됩니다.

보유 참조:

- UIManager UI
- GameManager Game
- SceneFlowManager SceneFlow
- MapManager Map
- CameraManager Camera
- LightingManager Lighting
- AdventureRunManager AdventureRun
- PlayerManager Player

주의:

- PlayerManager는 MonoBehaviour가 아니라 **Managers.Awake()**에서 **new PlayerManager()**로 생성됩니다.
- 나머지 Manager들은 Scene 또는 DDOL 오브젝트에 붙은 MonoBehaviour 참조입니다.

3. Game / Scene Flow

Assets/Scripts/Managers/GameManager/GameManager.cs

게임의 기본 상태를 관리합니다.

상태:

- Playing
- Paused
- GameOver

주요 메서드:

- `Pause()`: Playing일 때만 Paused로 전환하고 `Time.timeScale = 0f`
- `Resume()`: Paused일 때만 Playing으로 전환하고 `Time.timeScale = 1f`
- `Reset()`: 상태를 Playing으로 초기화
- `TriggerGameOver()`: 상태를 GameOver로 전환하고 시간 정지

현재 구조에서 Clear 상태는 GameManager가 아니라 `AdventureRunManager.RunState.Completed`로 표현합니다.

Assets/Scripts/Managers/GameManager/SceneFlowManager.cs

씬 전환 전담 Manager입니다.

상태:

- Title
- InGame
- Loading

주요 메서드:

- `StartGame()`
- `RestartGame()`
- `GoToTitle()`
- 내부 코루틴 `TransitionTo(sceneName, nextState)`

역할:

- UI Overlay fade in/out
- Loading UI 표시
- `SceneManager.LoadSceneAsync` 실행
- Game Scene 로드 완료 후 `AdventureRunManager.StartRun()` 호출
- Title 복귀 처리

중요한 책임 분리:

- `SceneFlowManager`: 씬을 이동시킵니다.
- `AdventureRunManager`: 한 판의 시작, 성공, 실패, 정리를 관리합니다.

4. Adventure Run

Assets/Scripts/Managers/AdventureRunManager/AdventureRunManager.cs

한 판의 런 루프를 관리하는 중심 클래스입니다.

상태:

- None
- Starting
- Playing

- Completed
- Failed

주요 메서드:

- StartRun()
- EndRun()
- FailRun()
- ClearRun()

StartRun() 흐름:

- State = Starting
- GameManager.Reset()
- MapManager.GenerateMap()
- GeneratedMapInfo.playerSpawnPosition 기준으로 Player 생성
- NotifyDialogue에 현재 InGameUI.Dialogue 등록
- Camera Follow 타겟을 Player로 연결
- 2D Global Light 준비
- State = Playing

EndRun() / FailRun():

- 두 메서드는 내부 FinishRun()을 공유합니다.
- EndRun()은 클리어 메시지를 재생한 뒤 Title로 복귀합니다.
- FailRun()은 GameManager.TriggerGameOver()를 호출하고 GameOver 메시지를 재생한 뒤 Title로 복귀합니다.

정리 대상:

- NotifyDialogue.UnregisterDialogue()
- PlayerSpawnManager.ClearCurrentPlayer()
- MapManager.ClearMap()
- CurrentMapInfo = null

Assets/Scripts/Managers/AdventureRunManager/GoalTrigger.cs

EndLine Chunk 또는 Goal Prefab에 붙는 클리어 트리거입니다.

모드:

- Touch: Player가 Trigger Collider에 닿으면 클리어
- Interact: IInteractable.Interact() 호출 시 클리어

호출 결과:

```
GoalTrigger.Reach()
-> Managers.Instance.AdventureRun.EndRun()
```

Assets/Scripts/Managers/AdventureRunManager/NotifyDialogue.cs

Clear, GameOver 같은 공지성 다이얼로그를 관리합니다.

보유 데이터:

- _clearMessage
- _gameOverMessage

구조:

- 직접 재생 로직을 갖지 않고 DialogueSequencePlayer를 사용합니다.

- `AdventureRunManager.StartRun()` 시점에 `UIDialogueController`를 등록합니다.
- 런 종료 정리 시 등록을 해제합니다.

5. Runtime Map Generation

경로는 현재 프로젝트 오타를 유지해 `MapManger`입니다.

`Assets/Scripts/Managers/GameManager/MapManger/MapManager.cs`

런타임 맵 생성의 외부 진입점입니다.

주요 필드:

- `_generationSettings`
- `_generateOnStart`

주요 메서드:

- `GenerateMap()`
- `ClearMap()`

현재 데모에서는 `AdventureRunManager.StartRun()`에서 `GenerateMap()`을 호출합니다.

`Assets/Scripts/Managers/GameManager/MapManger/GridMapGenerationSettings.cs`

`GridChunk` 맵 생성을 위한 `ScriptableObject`입니다.

주요 값:

- `GridChunkDatabase database`
- `int contentChunkCount`
- `int maxThreeDirCount`
- `bool useRandomSeed`
- `int randomSeed`
- `GridSpawnPrefabSet spawnPrefabs`

`ResolveSeed()`:

- `useRandomSeed == true: System.Environment.TickCount`
- `useRandomSeed == false: randomSeed`

현재 사용 Asset:

- `Assets/Prefabs/@Data/Map/GridChunkSettings/GridMapGenerationSettings1.asset`

`Assets/Scripts/Managers/GameManager/MapManger/GridChunkRuntimeGenerator.cs`

`EditorWindow` 없이 런타임에서 `GridChunk` 맵을 생성하는 정적 클래스입니다.

생성 흐름:

- 기존 `Generated Map Root` 삭제
- `Start EndLine Chunk` 선택 및 배치
- `Content Chunk`를 백트래킹 기반으로 배치
- `Side Branch` 생성
- `Dead End Cap` 배치
- `End EndLine Chunk` 배치
- `GeneratedMapInfo`를 생성된 맵 루트에 추가
- `playerSpawnPosition, endPosition` 기록

`Assets/Scripts/Managers/GameManager/MapManger/GridChunkGenerationContext.cs`

`Instantiate / Destroy`를 감싼 컨텍스트입니다.

런타임 생성 방식과 테스트용 생성 방식을 나눌 수 있는 확장 지점입니다.

Assets/Scripts/Managers/GameManager/MapManger/RuntimeGridPlacementUtility.cs

GridChunk 배치 계산 유틸리티입니다.

역할:

- 입구/출구 연결 계산
- 회전 및 위치 계산
- 점유 셀 계산
- 겹침 검사
- 배치 프레임 보조 타입 제공

Assets/Scripts/Managers/GameManager/MapManger/GridSpawnPrefabSet.cs

SpawnKind별 Prefab 매핑입니다.

현재는 Monster Prefab 연결이 주요 사용처입니다.

6. Map Data

Assets/Scripts/Map/GridChunkDB/GridChunkData.cs

GridChunk Prefab에 붙는 데이터 컴포넌트입니다.

주요 타입:

- GridChunkType
- GridEndLineRole
- GridEntranceSlot
- SpawnKind
- GridSpawnResolver
- GridEntranceSlotResolver

역할:

- 청크 타입 정의
- EndLine의 Start / End / Normal 역할 정의
- 입구 슬롯 정보 제공
- SpawnKind별 스폰 지점 제공

Assets/Scripts/Map/GridChunkDB/GridChunkDatabase.cs

GridChunk 후보 목록을 보관하는 ScriptableObject입니다.

런타임 생성기는 이 Database에서 타입별 후보를 가져옵니다.

Assets/Scripts/Map/GridChunkDB/GridChunkDatabaseEntry.cs

Database 내부에 들어가는 개별 청크 정보입니다.

Assets/Scripts/Map/GridChunkDB/GeneratedMapInfo.cs

생성된 맵 루트에 붙는 결과 정보 컴포넌트입니다.

보관 값:

- playerSpawnPosition
- endPosition

Assets/Scripts/Map/ChunkDatabase/*

이전 Assemble Chunk 방식의 데이터 구조입니다.

현재 데모 루프의 핵심은 `GridChunkDB`와 `GridChunkRuntimeGenerator`입니다.

7. Player / FSM

`Assets/Scripts/FSM/Player/@Hub/PlayerController.cs`

Player 전용 컨트롤러입니다.

`GroundedAgentController`를 상속합니다.

역할:

- `Stamina` 초기화
- `AgentImpactHandler` 초기화
- `PlayerStateFactory`로 FSM 상태 생성
- 공격 입력 처리
- 사망 애니메이션 완료 시 `AdventureRunManager.FailRun()` 호출

`Assets/Scripts/FSM/Player/Input/PlayerInput.cs`

Unity Input System 기반 Player 입력 컴포넌트입니다.

구현 인터페이스:

- `IAgentMovementInput`
- `IAgentJumpInput`
- `IAgentCombatInput`
- `IAgentInteractionInput`

`PlayerManager.Register(input)`로 현재 입력 컴포넌트를 등록합니다.

Dialogue 중에는 `UIDialogueController`가 입력 차단에 사용합니다.

`Assets/Scripts/FSM/Player/Input/PlayerInputCommands.inputactions`

Input System 액션 에셋입니다.

`Assets/Scripts/FSM/Player/Input/PlayerInputCommands.cs`

Input System이 생성한 코드입니다.

일반적으로 직접 수정하지 않는 편이 좋습니다.

`Assets/Scripts/FSM/Player/PlayerState/PlayerStateFactory.cs`

Player FSM 상태를 생성합니다.

현재 Player는 기본 `Agent/Grounded` 상태를 조합해 사용합니다.

`Assets/Scripts/FSM/Player/PlayerInteractionHandler.cs`

주변 `IInteractable`을 탐색하고 상호작용 입력 시 `Interact()`를 호출합니다.

8. Agent / Grounded Base

`Assets/Scripts/FSM/@Base/StateMachine/*`

FSM 공통 기반입니다.

포함:

- `IState`
- `IAgentState`
- `StateMachine<T>`

- `StateType`
- `AnimEventType`
- `InputKeyType`

Assets/Scripts/FSM/Agent/@Hub/AgentController.cs

Agent 계열의 최상위 컨트롤러입니다.

Health, Input, Animation 이벤트를 받아 상태 머신과 핸들러에 전달합니다.

Assets/Scripts/FSM/Agent/@Hub/Animator.cs

Animator 파라미터 제어 래퍼입니다.

Assets/Scripts/FSM/Agent/StateControl/*

Agent FSM 상태 기반과 공통 상태입니다.

주요 상태:

- `IdleState`
- `MoveState`
- `AttackState`
- `HitState`
- `DeathState`

Assets/Scripts/FSM/Agent/StateControl/*

2D Grounded Agent 확장 계층입니다.

주요 클래스:

- `GroundedAgentController`
- `GroundedAgentStateBase`
- `GroundedIdleState`
- `GroundedMoveState`
- `JumpState`
- `FallState`
- `GroundDetector`
- `GroundedAgentInputHandler`

역할:

- 지면 판정
- 점프/낙하 상태
- 2D 플랫폼 이동 상태

Assets/Scripts/FSM/Agent/Handler/*

Agent 동작을 실제로 처리하는 핸들러 계층입니다.

주요 클래스:

- `AgentInputHandler`
- `AgentMovementHandler2D`
- `AgentCombatHandler`
- `AgentHealthHandler`
- `AgentImpactHandler`
- `AgentAnimationEventProxy`

Assets/Scripts/FSM/Agent/Combat/*

전투 자원과 피격 처리입니다.

주요 클래스:

- Health
- Stamina
- IDamageable

Assets/Scripts/FSM/Agent/SOData/*

Agent 설정용 ScriptableObject입니다.

주요 클래스:

- AgentMotorData
- AgentStatData
- AnimationDataSO
- AttackData

9. NPC / Monster AI

Assets/Scripts/FSM/NPC/AIMonstor/@Hub/MonsterController.cs

Monster 전용 컨트롤러입니다.

AgentController 기반으로 동작합니다.

Assets/Scripts/FSM/NPC/AIMonstor/Input/AIMonsterInput.cs

Monster AI가 Agent 입력 인터페이스로 이동/공격 값을 공급하는 입력 어댑터입니다.

Assets/Scripts/FSM/NPC/AIMonstor/MonsterState/MonsterStateFactory.cs

Monster FSM 상태 생성 클래스입니다.

Assets/Scripts/FSM/NPC/AIMonstor/@Behavior/*

Monster Behavior Tree/AI 관련 스크립트와 액션입니다.

주요 파일:

- OrcBrain.cs
- MonsterTurnAction.cs
- MonsterMove2DAction.cs
- MonsterChaseTargetAction.cs
- MonsterAttackAction.cs

Assets/Scripts/FSM/NPC/@Detector/*

AI 판단에 사용하는 감지 컴포넌트입니다.

주요 클래스:

- PlayerDetector
- WallDetector
- JumpFloorDetector
- FieldOfView2D

Assets/Scripts/FSM/NPC/AIPlayer/*

AI Player 테스트 또는 시뮬레이션용 구조로 보입니다.

현재 Adventure 데모의 핵심 루프에서는 Monster 쪽이 더 직접적입니다.

10. Player Data / Inventory / Equipment

Assets/Scripts/Managers/PlayerManager/PlayerManager.cs

런타임 Player 관련 전역 상태를 보관합니다.

보관 값:

- InventorySystem Inventory
- PlayerEquipment Equipment
- PlayerController CurrentPlayer
- PlayerInput Input

이벤트:

- OnPlayerChanged

역할:

- Player 생성 시 등록
- Player 삭제 시 해제
- HUD가 런타임 생성 Player를 다시 바인딩할 수 있게 알림

Assets/Scripts/Managers/PlayerManager/PlayerSpawnManager.cs

Player Prefab 생성과 삭제를 담당합니다.

흐름:

```
Spawn(position)
-> ClearCurrentPlayer()
-> Instantiate(_playerPrefab)
-> Managers.Instance.Player.RegisterPlayer(CurrentPlayer)
```

Assets/Scripts/Managers/PlayerManager/InventorySystem.cs

아이템 목록과 아이템 추가/삭제 이벤트를 관리합니다.

이벤트:

- OnChanged
- OnItemAdded

Assets/Scripts/Managers/PlayerManager/PlayerEquipment.cs

장비 슬롯별 장착 상태와 보유 장비 목록을 관리합니다.

슬롯:

- None
- Weapon
- Armor
- Accessory

11. UI

Assets/Scripts/Managers/UIManager/UIManager.cs

전역 UI 진입점입니다.

보유 참조:

- `UIOverlayController Overlay`
- 현재 Scene의 `InGameUI`

`UIDialogueController`는 `UIManager`가 직접 보관하지 않고 `InGameUI` 내부에서 관리합니다.

Assets/Scripts/Managers/UIManager/Title/TitleScreen.cs

Title 화면 버튼 이벤트 처리 클래스입니다.

- Start Button: `Managers.Instance.SceneFlow.StartGame()`
- Quit Button: `Application.Quit()`

Assets/Scripts/Managers/UIManager/UIOverlayController.cs

화면 전환과 공통 오버레이를 담당합니다.

기능:

- Fade In
- Fade Out
- Damage Flash
- Pause Dim
- Loading UI 표시
- Overlay Reset

Assets/Scripts/Managers/UIManager/LoadingUI.cs

로딩 진행률 UI입니다.

Assets/Scripts/Managers/UIManager/InGameUI/InGameUI.cs

Game Scene UI 루트입니다.

보유 참조:

- `UIHUD HUD`
- `UIDialogueController Dialogue`
- `UINotificationController Notification`
- `UIMenuTabController MenuTabController`

역할:

- `Awake()`에서 `Managers.Instance.UI.Register(this)`
- `OnDestroy()`에서 등록 해제
- 아이템 획득 알림 표시
- HUD 표시/숨김과 Pause/Resume 연결

Assets/Scripts/Managers/UIManager/InGameUI/Dialogue/UIDialogueController.cs

대화창 UI 제어 클래스입니다.

역할:

- Speaker/Text 표시
- 다음 버튼 또는 입력으로 Advance 호출
- Open 시 PlayerInput 차단 및 HUD 숨김
- Close 시 PlayerInput 복구 및 HUD 표시

Assets/Scripts/Managers/UIManager/InGameUI/Dialogue/DialogueSequencePlayer.cs

`DialogueDataSO`를 순서대로 재생하는 일반 C# 클래스입니다.

특징:

- MonoBehaviour가 아닙니다.
- RegisterDialogue(UIDialogueController)로 현재 Scene의 Dialogue를 등록합니다.
- Play(data, onFinish)가 성공 여부를 bool로 반환합니다.
- Advance, Finish, Cancel 흐름을 공통화합니다.

Assets/Scripts/Managers/UIManager/InGameUI/HUD/*

HUD 관련 UI입니다.

주요 클래스:

- UIHUD
- HealthView
- HealthViewModel
- HealthViewComposition
- StaminaView
- StaminaViewModel
- StaminaViewComposition
- MapScreen

Health/Stamina Composition은 `PlayerManager.OnPlayerChanged`를 구독해 런타임 생성 Player를 바인딩합니다.

Assets/Scripts/Managers/UIManager/InGameUI/Menu/*

인게임 메뉴와 탭 화면입니다.

주요 클래스:

- UIMenuTabController
- UITab
- InventoryScreen
- InventorySlot
- UIItemDetailPanel
- EquipmentScreen
- UIEquipArea
- EquipSlotButton
- EquipDetail
- SettingsScreen

UIMenuTabController는 메뉴 열기/닫기, 탭 버튼 스프라이트 변경, 화면 전환을 담당합니다.

12. Interaction / Dialogue / Item

Assets/Scripts/Interaction/IInteractable.cs

상호작용 가능한 오브젝트의 공통 인터페이스입니다.

Assets/Scripts/Interaction/Dialogue/DialogueDataSO.cs

대화 데이터 ScriptableObject입니다.

보관 값:

- Speaker Name
- Lines

Assets/Scripts/Interaction/Dialogue/NPCDialogue.cs

NPC용 상호작용 다이얼로그 컴포넌트입니다.

`IInteractable`을 구현하고 `DialogueSequencePlayer`를 사용합니다.

Assets/Scripts/Interaction/Item/ItemPickup.cs

아이템 획득 상호작용 컴포넌트입니다.

획득 시 `PlayerManager.Inventory`에 아이템을 추가하는 구조입니다.

Assets/Scripts/Managers/GameData/ItemData.cs

아이템 `ScriptableObject`입니다.

`Inventory`, `Equipment`, `UI Item Detail`에서 사용하는 데이터입니다.

13. Projectile / Trap

Assets/Scripts/Projectile/TrapBullet/*

트랩 탄환 관련 런타임 스크립트입니다.

주요 클래스:

- `TrapBulletFactory`
- `TrapBulletController`
- `PooledTrapBullet`

Assets/Scripts/Managers/ProjectileManager/TrapBulletPoolManager.cs

Trap Bullet 풀을 관리하는 Manager입니다.

14. Camera / Lighting / Map Visual

Assets/Scripts/Managers/CameraManager/CameraManager.cs

Main Camera 탐색 또는 Prefab 생성, Follow 컴포넌트 확보, Player 타겟 바인딩을 담당합니다.

Assets/Scripts/Managers/CameraManager/CameraFollow2D.cs

`LateUpdate`에서 타겟을 부드럽게 따라가는 2D 카메라 Follow 컴포넌트입니다.

Assets/Scripts/Managers/LightingManager/LightingManager.cs

2D Global Light를 찾거나 생성해 런 시작 시 조명을 보장합니다.

Assets/Scripts/Map/ParallaxLayer.cs

배경 Parallax 연출용 컴포넌트입니다.

15. Editor 전용 스크립트

`Assets/Scripts/Editor/*` 아래 스크립트는 런타임 빌드에 포함되지 않는 Editor 도구입니다.

Map Generator

주요 파일:

- `GridChunkMapGeneratorWindow.cs`
- `GridChunkDatabaseBuilder.cs`
- `GridChunkDataEditor.cs`
- `ChunkMapGeneratorWindow.cs`
- `ChunkDatabaseBuilder.cs`
- `ChunkLinearGenerator.cs`
- `ChunkBranchingGenerator.cs`

- `ChunkPlacementUtility.cs`

역할:

- GridChunk 데이터 작성
- Database 생성
- Editor 미리보기/배치 보조

런타임 맵 생성은 `Assets/Scripts/Managers/GameManager/MapManger/*` 쪽을 기준으로 봐야 합니다.

Agent Combat Editor

주요 파일:

- `AgentCombatEditor.cs`
- `AgentCombatHandlerEditor.cs`

역할:

- Agent 전투 데이터와 핸들러 Inspector/Editor 보조

16. 현재 데모에서 가장 중요한 연결점

다른 AI가 작업을 이어갈 때 우선 확인할 파일:

- `Assets/Scripts/Managers/Managers.cs`
- `Assets/Scripts/Managers/GameManager/SceneFlowManager.cs`
- `Assets/Scripts/Managers/AdventureRunManager/AdventureRunManager.cs`
- `Assets/Scripts/Managers/GameManager/MapManger/MapManager.cs`
- `Assets/Scripts/Managers/GameManager/MapManger/GridChunkRuntimeGenerator.cs`
- `Assets/Scripts/Managers/PlayerManager/PlayerSpawnManager.cs`
- `Assets/Scripts/Managers/PlayerManager/PlayerManager.cs`
- `Assets/Scripts/FSM/Player/@Hub/PlayerController.cs`
- `Assets/Scripts/Managers/CameraManager/CameraManager.cs`
- `Assets/Scripts/Managers/LightingManager/LightingManager.cs`
- `Assets/Scripts/Managers/UIManager/InGameUI/InGameUI.cs`
- `Assets/Scripts/Managers/UIManager/InGameUI/Dialogue/DialogueSequencePlayer.cs`
- `Assets/Scripts/Managers/AdventureRunManager/NotifyDialogue.cs`
- `Assets/Scripts/Managers/AdventureRunManager/GoalTrigger.cs`

17. 설계상 주의사항

- `SceneFlowManager`와 `AdventureRunManager`의 책임을 섞지 않는 것이 좋습니다.
- `GameManager`는 현재 게임 상태와 시간 제어만 담당합니다.
- `Clear`는 아직 `GameManager.GameState`가 아니라 `AdventureRunManager.RunState.Completed`로 처리합니다.
- Player 생성/삭제는 `PlayerSpawnManager`를 통해 처리합니다.
- 런타임 생성 Player 참조는 `PlayerManager.CurrentPlayer`를 기준으로 가져옵니다.
- Scene 전환 중 `InGameUI` 참조는 바뀔 수 있으므로, Dialogue는 런 시작 시 등록하고 런 정리 시 해제합니다.
- `PlayerInputCommands.cs`는 Input System generated code이므로 직접 수정하지 않는 편이 좋습니다.
- Editor 폴더 코드는 런타임 데모 루프와 분리해서 판단해야 합니다.
- `MapManger` 폴더명은 오타처럼 보이지만 현재 프로젝트 경로이므로 파일 이동 전 참조 영향을 확인해야 합니다.

18. 다음 확장 후보

- GameOver 전용 `DialogueDataSO` Asset 연결

- Goal prefab의 Touch/Interact 모드별 테스트
- Pause/Menu 입력 흐름 정리
- Inventory/Equipment 실제 장비 효과 연결
- Monster 사망/보상 루프 연결
- 다음 맵 또는 스테이지 진행 구조 추가
- 런 결과 UI 추가